

Advanced Supervised Learning

Models and Problems

Mehmet Mercangöz

AI and AIS Summer School July/Aug 2026

① Tree models

- decision trees, ensembles, boosting, XGBoost

② Neural networks

- layers, activations, backpropagation, training

③ Convolutional neural networks

- spatial structure, filters, feature maps

④ Graph neural networks

- message passing on graph-structured data

⑤ Recurrent neural networks

- sequences, backpropagation through time

⑥ Gated sequence models

- LSTMs and GRUs

⑦ Transformers

- attention, multi-head attention, architecture

⑧ Model choice

- matching models to data and tasks

The Big Picture: Models for Different Data Structures

Data type	Model family	Typical supervised task
Tabular data	Trees, boosting, MLPs	Predict risk, quality, demand, failure
Images and grids	CNNs	Image classification, defect detection
Graphs and networks	GNNs	Node, edge, or graph prediction
Sequences	RNNs, LSTMs, GRUs, Transformers	Time series, text, event sequences
Sets or tokens	Transformers	Classification, generation, sequence-to-sequence



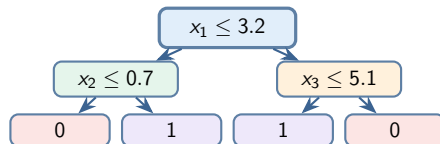
Main idea

Choose an architecture whose structure matches the structure of the data.

Decision Trees: The Basic Idea

A decision tree makes predictions by asking a sequence of simple questions.

- Internal nodes contain tests.
- Branches correspond to test outcomes.
- Leaves contain predictions.
- The path from root to leaf defines a rule.



For tabular data, a split often has the form

$$x_j \leq s.$$

Trees Partition the Input Space

Each split divides the feature space into simpler regions.

For two input features, a tree produces axis-aligned regions.

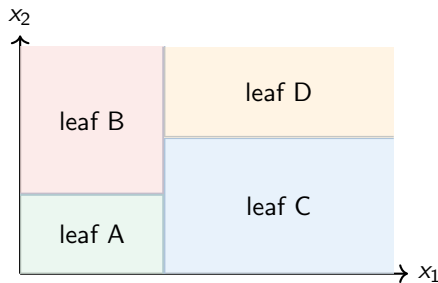
Within each leaf, the prediction is simple:

$$\hat{y} = \text{average target in the leaf}$$

for regression, or

$$\hat{y} = \text{majority class in the leaf}$$

for classification.



How Does a Tree Choose a Split?

A split is chosen to make the child nodes more homogeneous than the parent node.

Regression trees

A common split criterion is reduction in squared error:

$$\sum_{i \in P} (y_i - \bar{y}_P)^2 - \sum_{i \in L} (y_i - \bar{y}_L)^2 - \sum_{i \in R} (y_i - \bar{y}_R)^2.$$

Classification trees

A common impurity measure is Gini impurity:

$$G = 1 - \sum_{k=1}^K p_k^2,$$

where p_k is the fraction of class k in the node.

Intuition

Good splits produce leaves that are internally simple.

Worked Example: Training a Classification Tree

We train a tree from the following data:

x	1	2	3	4	5
y	0	0	1	1	0

The tree starts with all observations in one root node:

$$\mathcal{D}_{\text{root}} = \{(1, 0), (2, 0), (3, 1), (4, 1), (5, 0)\}.$$

Sort the observed feature values:

$$1, 2, 3, 4, 5.$$

Use midpoints between consecutive observed values:

$$s_1 = \frac{1+2}{2} = 1.5, \quad s_2 = \frac{2+3}{2} = 2.5,$$

$$s_3 = \frac{3+4}{2} = 3.5, \quad s_4 = \frac{4+5}{2} = 4.5.$$

Root Node: Enumerating All Candidate Splits

For each threshold s , split using

$$L(s) = \{i : x_i \leq s\}, \quad R(s) = \{i : x_i > s\}.$$

The weighted Gini score is

$$J(s) = \frac{|L|}{5} G(L) + \frac{|R|}{5} G(R).$$

s	y_L	y_R	$J(s)$
1.5	[0]	[0, 1, 1, 0]	$\frac{1}{5}(0) + \frac{4}{5}(0.5) = 0.400$
2.5	[0, 0]	[1, 1, 0]	$\frac{2}{5}(0) + \frac{3}{5}(\frac{4}{9}) = 0.267$
3.5	[0, 0, 1]	[1, 0]	$\frac{3}{5}(\frac{4}{9}) + \frac{2}{5}(0.5) = 0.467$
4.5	[0, 0, 1, 1]	[0]	$\frac{4}{5}(0.5) + \frac{1}{5}(0) = 0.400$

The smallest score is obtained at

$$s^* = 2.5.$$

So the first rule is

$x \leq 2.5?$

After the Root Split: Recurse on the Child Nodes

The root split is

$$x \leq 2.5?$$

left node	right node
$[0, 0]$	$[1, 1, 0]$

Left child (done)

$$y = [0, 0]$$

This node is pure:

$$G = 0.$$

So it becomes a leaf:

$$\hat{y} = 0$$

Right child (continues)

x	3	4	5
y	1	1	0

Generate new thresholds:

$$\frac{3+4}{2} = 3.5, \quad \frac{4+5}{2} = 4.5.$$

Training the Right Child

At the right child, the candidate thresholds are

$$s = 3.5, \quad s = 4.5.$$

s	y_L	y_R	$J(s)$
3.5	[1]	[1, 0]	$\frac{1}{3}(0) + \frac{2}{3}(0.5) = 0.333$
4.5	[1, 1]	[0]	$\frac{2}{3}(0) + \frac{1}{3}(0) = 0$

The smallest score is obtained at

$$s^* = 4.5.$$

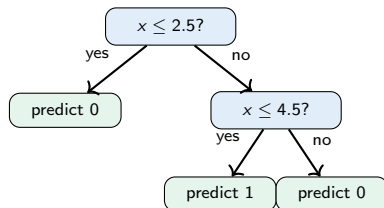
So the right-child rule is

$$x \leq 4.5?$$

Both resulting child nodes are pure:

$$[1, 1] \rightarrow \hat{y} = 1, \quad [0] \rightarrow \hat{y} = 0.$$

Final Trained Tree



Learned prediction rule

$$\hat{y}(x) = \begin{cases} 0, & x \leq 2.5, \\ 1, & 2.5 < x \leq 4.5, \\ 0, & x > 4.5. \end{cases}$$

Interpretation

The tree has divided the input space into three regions, with one constant prediction in each region.

Tree Hyperparameters Control Complexity

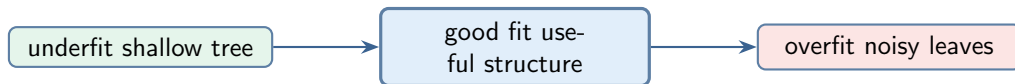
A tree can overfit if it keeps splitting until every leaf is very small.

Controls on tree size

- maximum depth;
- minimum samples per leaf;
- minimum improvement required for a split;
- pruning after growing the tree.

Trade-off

- shallow tree: interpretable, but may underfit;
- deep tree: flexible, but may overfit;
- ensembles reduce the instability of single trees.

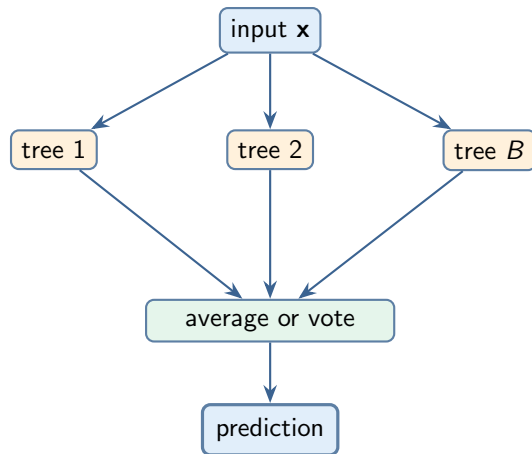


Random Forests: Reduce Variance by Averaging Trees

A single decision tree can be sensitive to small data changes. A random forest averages many different trees.

- Train each tree on a bootstrap sample of the data.
- At each split, consider only a random subset of features.
- Average predictions for regression.
- Vote or average probabilities for classification.

$$\hat{f}_{\text{forest}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x}).$$

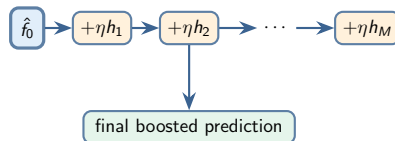


Gradient Boosting: Add Trees Sequentially

Boosting builds an ensemble one model at a time. Each new tree tries to correct what the current ensemble gets wrong.

$$\hat{f}_M(\mathbf{x}) = \hat{f}_0(\mathbf{x}) + \sum_{m=1}^M \eta h_m(\mathbf{x}).$$

- h_m is the tree added at stage m .
- η is the learning rate.
- Trees are usually shallow.
- The ensemble can be very accurate on tabular data.



Gradient Boosting as Functional Gradient Descent

We want to minimise a loss over functions:

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{f}(\mathbf{x}_i)) .$$

At boosting step m , compute pseudo-residuals

$$r_i^{(m)} = - \left. \frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i} \right|_{\hat{y}_i = \hat{f}_{m-1}(\mathbf{x}_i)} .$$

Then fit a tree $h_m(\mathbf{x})$ to these residual-like targets.

Intuition

Ordinary gradient descent changes parameters. Gradient boosting changes the prediction function by adding a new tree in a descent direction.

XGBoost: Regularised Gradient Boosted Trees

XGBoost is an efficient and regularised implementation of gradient boosted decision trees. The model is an additive ensemble:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F}.$$

A typical objective is

$$\mathcal{J} = \sum_i \ell(y_i, \hat{y}_i) + \sum_k \Omega(f_k).$$

What makes it powerful?

- second-order loss approximation;
- regularisation on tree complexity;
- shrinkage and subsampling;
- efficient split search;
- strong performance on tabular data.

Tree Models: Strengths and Weaknesses

Strengths

- handle nonlinear feature interactions;
- work well on tabular data;
- little need for feature scaling;
- can handle mixed feature types;
- often strong baselines.

Weaknesses

- single trees can be unstable;
- extrapolation outside training range is weak;
- very high-dimensional unstructured data may need deep learning;
- interpretation of large ensembles is less direct.

The Artificial Neuron: The Basic Building Block

Neural networks are built from simple processing units called artificial neurons, inspired loosely by biological brains.

A neuron performs two operations:

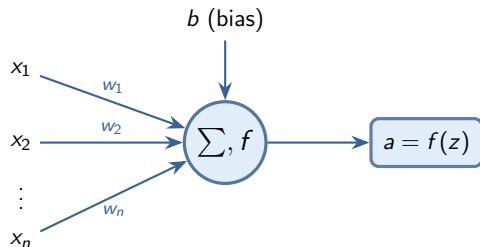
- 1 **Linear transformation:** Computes a weighted sum of inputs plus a bias term.

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

- 2 **Nonlinear activation:** Passes the result through an activation function to determine the output.

$$a = f(z)$$

Without the activation function, the entire network would collapse into a simple linear regression model.



Multi-Layer Perceptrons (Feedforward Networks)

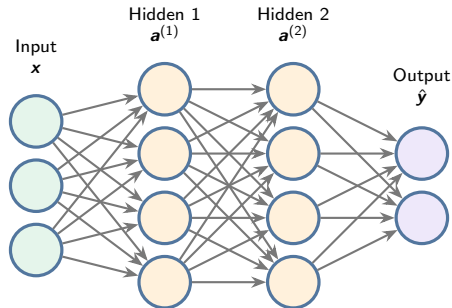
By connecting layers of neurons, we create a Multi-Layer Perceptron (MLP). Data flows in one direction: from inputs to outputs.

For a hidden layer l , the computation is written in vectorised form:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

- $\mathbf{W}^{(l)}$: Weight matrix for layer l .
- $\mathbf{b}^{(l)}$: Bias vector for layer l .
- $\mathbf{a}^{(0)}$ is the input vector $\mathbf{x}$.
- The final layer produces the prediction $\hat{y}$.

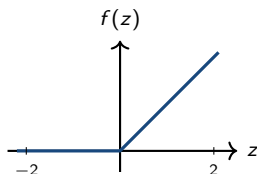


Common Activation Functions

ReLU

Rectified Linear Unit

$$\text{ReLU}(z) = \max(0, z)$$

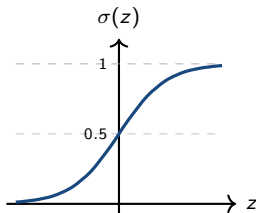


Common hidden-layer activation:
zero for negative inputs and linear for
positive inputs.

Sigmoid

Binary proba-
bility output

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

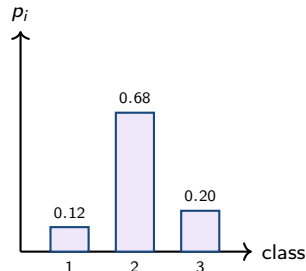


Maps a score to a value between 0
and 1; commonly used for binary
classification outputs.

Softmax

Multi-class prob-
ability output

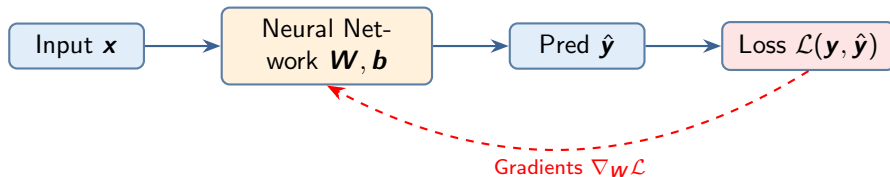
$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$



Converts several class scores into

Training: Forward and Backward Passes

Neural networks are trained iteratively using gradient descent. The process relies on two key passes through the network.



1. Forward Pass

Pass the input through the network layer by layer to generate a prediction $\hat{y}$. Compute the error using a Loss function (e.g., Cross-Entropy).

2. Backward Pass (Backpropagation)

Compute how much each weight contributed to the error using the chain rule of calculus. Pass these gradients backward through the layers.

Backpropagation and Parameter Updates

Once the gradients are calculated via backpropagation, we update the weights to minimise the loss.

The Chain Rule

To find how a weight w_{ij} in an earlier layer affects the final loss $\mathcal{L}$, we multiply the local gradients:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

Gradient Descent Update

We adjust the weights in the opposite direction of the gradient:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}}$$

where η is the learning rate.

Optimisers

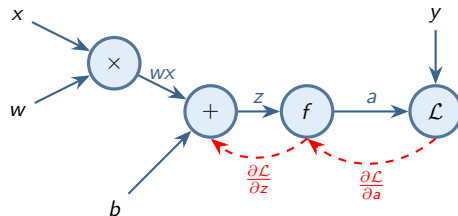
In practice, we use advanced variations of Stochastic Gradient Descent (SGD):

- **Momentum:** Accelerates gradients in consistent directions.
- **Adam:** Adapts the learning rate for each individual parameter automatically.

Backpropagation: The Computational Graph

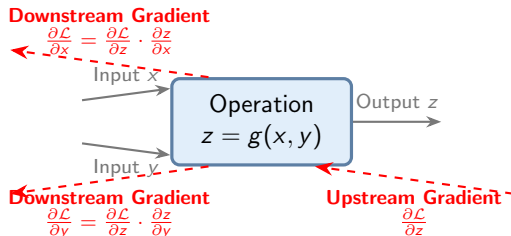
To understand backpropagation, it helps to view the neural network as a **computational graph**. Every mathematical operation is a node.

- **Forward Pass:** Compute the outputs of each node from left to right.
- We must cache the intermediate variables (like z) because they are required to compute the gradients later.
- **Backward Pass:** Compute the gradients from right to left using the chain rule.



The Chain Rule: Local vs. Upstream Gradients

At any given node in our graph, backpropagation relies on a simple principle: multiplying the **upstream gradient** by the **local gradient**.



The Mechanism

The node doesn't need to know about the whole network. It only needs to know its own local derivatives $(\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y})$ and multiply them by the error signal it receives from above $(\frac{\partial \mathcal{L}}{\partial z})$.

Zooming In: Gradients of a Single Neuron

Let's apply this to a single weight w_i in a neuron. The equations are:

$$z = \sum w_i x_i + b \quad \text{and} \quad a = f(z)$$

1. The Local Gradients:

- Derivative of the activation:

$$\frac{\partial a}{\partial z} = f'(z)$$

- Derivative of the sum w.r.t the weight:

$$\frac{\partial z}{\partial w_i} = x_i$$

- Derivative of the sum w.r.t the bias:

$$\frac{\partial z}{\partial b} = 1$$

2. The Full Chain Rule: Assuming we receive the upstream gradient $\frac{\partial \mathcal{L}}{\partial a}$:

$$\frac{\partial \mathcal{L}}{\partial w_i} = \underbrace{\frac{\partial \mathcal{L}}{\partial a}}_{\text{Upstream}} \cdot \underbrace{f'(z) \cdot x_i}_{\text{Local}}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \underbrace{\frac{\partial \mathcal{L}}{\partial a}}_{\text{Upstream}} \cdot \underbrace{f'(z) \cdot 1}_{\text{Local}}$$

The weight update is directly proportional to its input x_i . If the input to a synapse is zero, that specific weight's gradient is zero (it won't be updated).

Neural Networks: Strengths and Weaknesses

Strengths

- **Universal Approximators:** Can learn almost any continuous function given enough data and size.
- **Unstructured Data:** The foundation for state-of-the-art models in computer vision, NLP, and audio.
- **Feature extraction:** Layers automatically learn hierarchical representations (no manual feature engineering).
- **Scalability:** Performance continues to improve with massive datasets.

Weaknesses

- **Data Hungry:** Often require vast amounts of labeled data to prevent overfitting.
- **Computationally Heavy:** Training requires specialized hardware (GPUs/TPUs) and significant time.
- **Black-box nature:** Hard to interpret exactly *how* a network arrived at a specific decision.
- **Hyperparameters:** Sensitive to learning rates, architecture choice, and weight initialization.

Dropout: Randomly Remove Units During Training

Dropout is a regularisation method for neural networks.

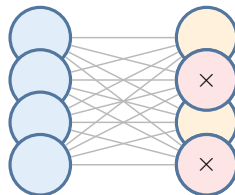
During training, some activations are randomly set to zero:

$$\tilde{\mathbf{a}}^{(\ell)} = \mathbf{m}^{(\ell)} \odot \mathbf{a}^{(\ell)},$$

where $\mathbf{m}^{(\ell)}$ is a random mask.

Effect

- prevents hidden units from relying too strongly on each other;
- acts like training many thinned networks;
- often reduces overfitting.



some units dropped during training

Other Common Training Tools

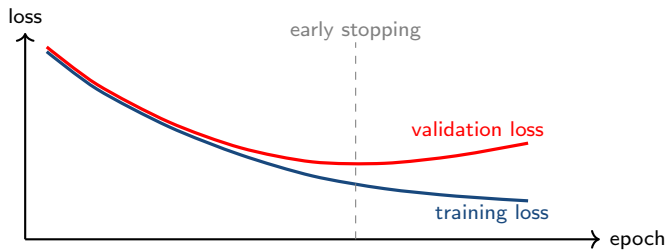
Tool	Purpose
Weight decay	Penalise large weights to reduce overfitting.
Early stopping	Stop training when validation loss stops improving.
Batch normalisation	Normalise intermediate activations to stabilise training.
Learning-rate schedule	Reduce or adapt the learning rate during training.
Adam optimiser	Adaptive gradient method widely used for neural networks.

Practical message

Neural network performance depends not only on the architecture, but also on the training procedure.

Training and Validation Curves

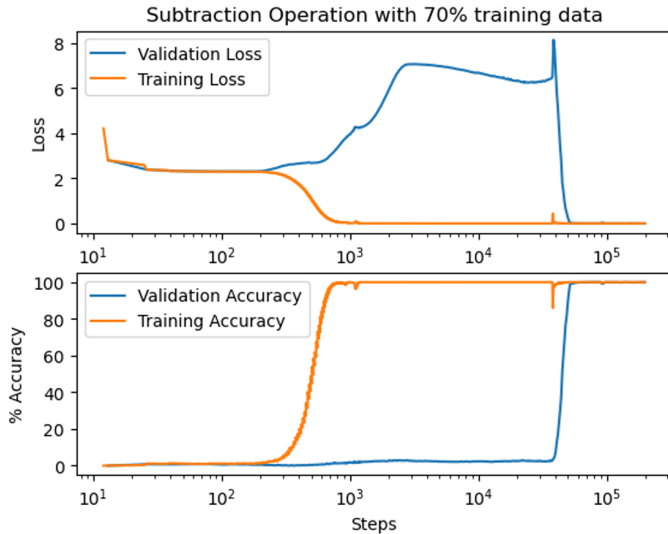
Training should be monitored using both training and validation loss.



Warning sign

If training loss keeps decreasing while validation loss increases, the model is overfitting.

Training and Validation Curves: Grokking



How to Think About Network Size

Network size controls model capacity.

Depth

Number of layers.

More depth allows more stages of representation learning.

Width

Number of units per layer.

More width allows more features to be represented at each stage.

Trade-off

- too small: underfitting;
- too large: overfitting or harder training;
- more data usually supports larger models;
- validation performance guides the choice.

Start simple, verify the pipeline, increase model capacity if needed.

Convolutional Neural Networks

To process an image with a standard Multi-Layer Perceptron (MLP), we must first "flatten" the 2D grid of pixels into a 1D vector. This creates two major problems.

MLP vs. CNN Approach

	Fully connected	Con- nected	CNNs
Connections	Dense (all-to-all)		Sparse (local only)
Weights	Independent		Shared across space
Topology	1D vectors		2D/3D grids
Translation	Not invariant		Invariant

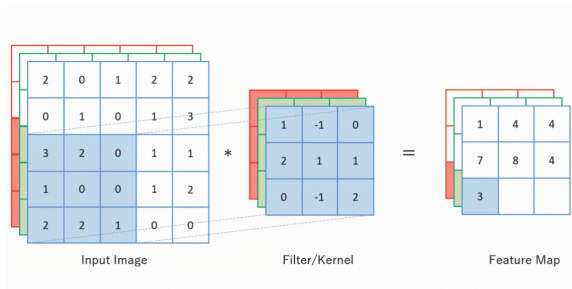
1.) Flattening destroys the 2D spatial relationships. The network has no built-in concept that a pixel at (x, y) is physically adjacent to $(x + 1, y)$. It must learn this from scratch.

2.) A small 200×200 RGB image has 120,000 input values. A fully connected layer with just 1,000 hidden neurons would require **120 million weights**. This easily leads to overfitting and massive memory costs.

Instead of connecting every input to every neuron, CNNs use **local receptive fields** and **parameter sharing**.

The Convolution Operation

A convolution layer slides a small matrix of weights—called a **filter** or **kernel**—over the input image. At each position, it computes a dot product between the filter and the local image patch.



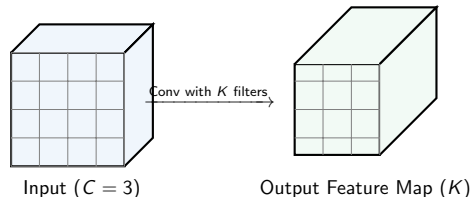
- **Local Connectivity:** The output pixel depends only on a 3×3 local patch of the input.
- **Weight Sharing:** The exact same 3×3 filter W is used across the entire image.
- **Feature Detection:** A filter might learn to detect vertical edges, horizontal lines, or specific color contrasts.

Channels and Multiple Filters

Images are rarely just 2D; they have depth (e.g., 3 color channels for RGB). Furthermore, a single filter can only detect one type of feature. Therefore, a convolutional layer applies **multiple filters**.

Input to Output Dimensions

- **Input Volume:** $H \times W \times C$
(e.g., $32 \times 32 \times 3$)
- **Filters:** We use K different filters. Each filter spans the full depth of the input ($F \times F \times C$).
- **Output Volume:** $H' \times W' \times K$. The depth of the output is equal to the number of filters!



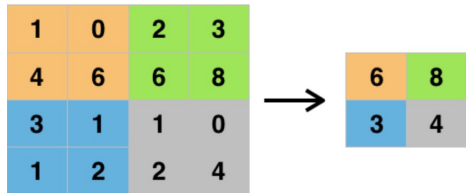
Hyperparameters of a Conv Layer

Filter Size (F): Usually 3×3 or 5×5 .

Stride (S): How many pixels the filter shifts.
Stride > 1 downsamples the image.

Pooling Layers: Spatial Downsampling

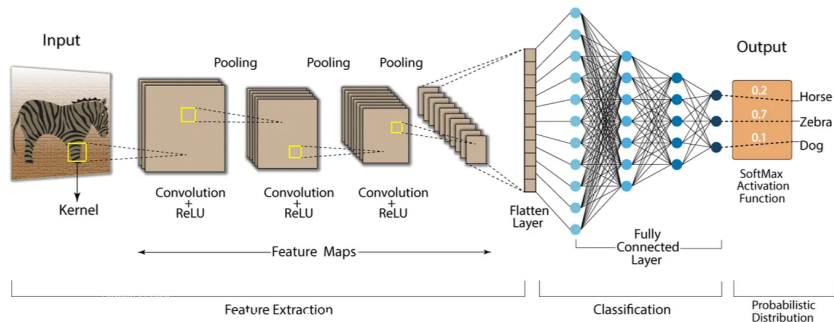
After a convolution and activation (usually ReLU), it is common to apply a **Pooling** layer. Pooling reduces the spatial dimensions, controlling overfitting and reducing computation.



- **Mechanism:** Slides a window (e.g., 2×2) and outputs only the maximum value in that window.
- **No Parameters:** Pooling layers have no weights to learn.
- **Translation Invariance:** If a feature shifts slightly in the input image, the max pooling output often remains exactly the same.

The Full CNN Architecture

A standard Convolutional Neural Network connects these building blocks to form a full pipeline: **Feature Extraction** followed by **Classification**.



Early layers learn simple features (edges, colors). Deeper layers combine these into complex textures (wheels, eyes). The fully connected layers use these high-level features to make a final prediction.

Figure from <https://medium.com/@saimoguloju2>

Graph-Structured Data

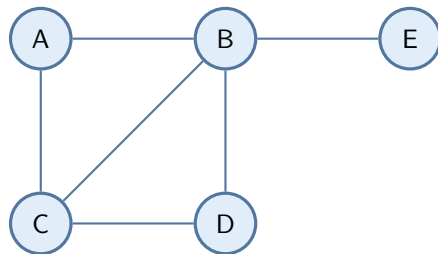
Some supervised learning problems are naturally defined on graphs.

A graph consists of nodes, edges, and possibly features:

$$G = (V, E), \quad \mathbf{h}_v \text{ for node } v, \quad \mathbf{e}_{uv} \text{ for edge } (u, v).$$

Examples:

- molecules: atoms and bonds;
- power networks: buses and lines;
- social networks: users and links;
- process flowsheets: units and streams;
- road networks: junctions and roads.



Graph Neural Networks: Message Passing

A graph neural network updates each node representation by aggregating information from neighbouring nodes.

$$\mathbf{m}_v^{(\ell)} = \text{AGGREGATE}^{(\ell)} \left(\left\{ \mathbf{h}_u^{(\ell-1)} : u \in \mathcal{N}(v) \right\} \right),$$

$$\mathbf{h}_v^{(\ell)} = \text{UPDATE}^{(\ell)} \left(\mathbf{h}_v^{(\ell-1)}, \mathbf{m}_v^{(\ell)} \right).$$

Intuition

After one layer, a node knows about its neighbours. After several layers, it has information from a larger neighbourhood.

Task type	Examples
Node prediction	classify a node, predict a node property, detect faulty equipment
Edge prediction	predict whether two nodes should be connected, link prediction, relation type
Graph prediction	classify a whole molecule, predict network risk, estimate graph-level performance

For graph-level prediction, node embeddings are combined using a readout function:

$$\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_v : v \in V\}), \quad \hat{y} = q(\mathbf{h}_G).$$

GNNs: Important Modelling Issues

Design choices

- node, edge, and graph features;
- aggregation function;
- number of message-passing layers;
- readout function;
- whether edge directions matter.

Challenges

- oversmoothing with too many layers;
- sparse labels;
- graph size variation;
- leakage through graph construction;
- explainability and uncertainty.

Modeling Sequences: The Vanilla RNN

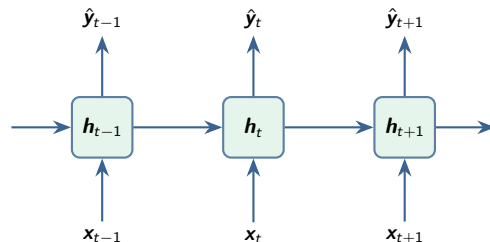
Standard MLPs and CNNs assume inputs are independent and of a fixed size. However, many real-world problems involve **sequential data** (time series, text, audio) where context from the past is crucial.

The Recurrent Loop

An RNN maintains a **hidden state** h_t that acts as a memory of previous inputs. At each time step t , it updates this state using the current input x_t and the previous state h_{t-1} :

$$h_t = \tanh(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$$

Unrolling an RNN through time



The Vanishing Gradient Problem

When backpropagating through many time steps, repeated multiplication of the weight matrix W_{hh} causes gradients to shrink to zero. Vanilla RNNs effectively “forget” long-term dependencies.

Long Short-Term Memory (LSTM) Networks

To solve the vanishing gradient problem, LSTMs introduce a more complex internal structure. Instead of a single hidden state, they use a **dual-state architecture** and **gates**.

1. The Cell State (C_t)

- The “long-term memory” or conveyor belt.
- Runs straight down the entire chain with only minor linear interactions.
- Information can flow along it easily, keeping gradients from vanishing.

2. The Hidden State (h_t)

- The “short-term memory”.
- Used to make the prediction at step t and passed to the next step to help compute the gates.

3. Gates (The Control Mechanism)

- Neural network layers that use a **sigmoid** (σ) activation function.
- Output values between 0 and 1.
- By multiplying a vector by a gate, the LSTM decides how much information to let through (0 = block, 1 = pass).

Deep Dive: The Forget Gate Mechanism

The first decision an LSTM makes is what information to throw away from the long-term Cell State ($\mathbf{C}_{t-1}$). This is controlled by the **Forget Gate layer**.

The gate looks at the previous hidden state and current input, and outputs a vector $\mathbf{f}_t$ with values between 0 and 1:

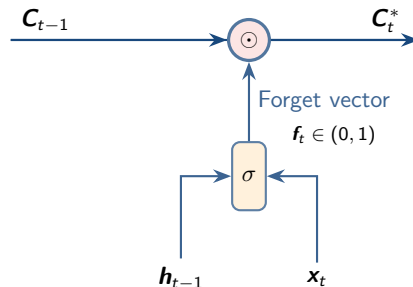
$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

We then apply element-wise multiplication $\odot$ to the old cell state:

$$\mathbf{C}_t^* = \mathbf{f}_t \odot \mathbf{C}_{t-1}$$

Intuition:

- If $f_{t,i} \approx 0$: “Completely forget” the past information in position i .
- If $f_{t,i} \approx 1$: “Completely keep” the past information.



Completing the LSTM Cell: Updating and Outputting

After the forget gate, the LSTM adds new information, updates the cell state, and produces the hidden state.

1. Select and create new information

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C).$$

2. Update the cell state

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

3. Produce the hidden-state output

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o),$$

$$h_t = o_t \odot \tanh(C_t).$$

Why are gradients preserved?

The cell-state update contains an additive path:

$$C_t = f_t \odot C_{t-1} + \text{new information}.$$

Therefore,

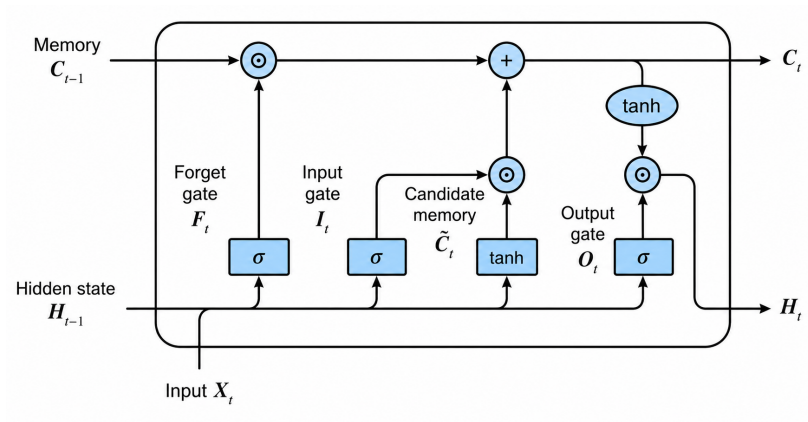
$$\frac{\partial C_t}{\partial C_{t-1}} = f_t.$$

When

$$f_t \approx 1,$$

the cell state and its gradient can be carried across many time steps with little decay.

LSTM Cell: Overview



Why Attention?

RNNs and LSTMs process a sequence step by step.

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T \longrightarrow \mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_T$$

This gives a useful memory, but long sequences are still difficult.

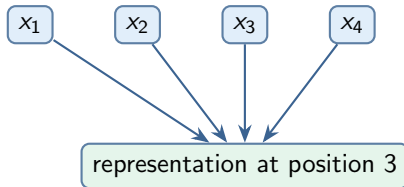
Problem

Important information may be far away in the sequence.

Early token but later prediction.

Attention idea

Instead of compressing the past into one state, let each position directly look at other relevant positions.



Attention as Weighted Averaging

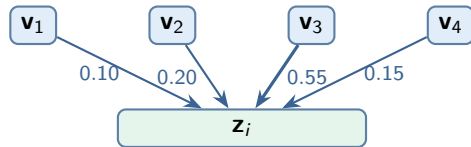
Attention builds a new representation by taking a weighted average of other representations.

$$\mathbf{z}_i = \sum_{j=1}^T \alpha_{ij} \mathbf{v}_j$$

$$\alpha_{ij} \geq 0, \quad \sum_{j=1}^T \alpha_{ij} = 1.$$

Meaning

For position i , the attention weights α_{ij} say how much position i uses information from position j .



Queries, Keys, and Values

Attention uses three learned vectors at each position.

$$\mathbf{q}_i = \mathbf{W}_Q \mathbf{x}_i, \quad \mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i, \quad \mathbf{v}_i = \mathbf{W}_V \mathbf{x}_i.$$

Query

What am I looking for?

$\mathbf{q}_i$

Key

What information do I contain?

$\mathbf{k}_j$

Value

What information do I pass on?

$\mathbf{v}_j$

Compatibility is measured by a dot product:

$$\text{score}_{ij} = \mathbf{q}_i^\top \mathbf{k}_j.$$

Scaled Dot-Product Attention

The attention scores are converted into probabilities using softmax.

$$\alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k})}{\sum_{\ell=1}^T \exp(\mathbf{q}_i^\top \mathbf{k}_\ell / \sqrt{d_k})}.$$

In matrix form:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

Role of the scaling

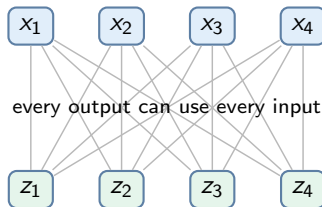
The factor $\sqrt{d_k}$ keeps dot products from becoming too large when the vector dimension is large.

Self-Attention

In self-attention, the queries, keys, and values all come from the same input sequence.

$$X \longrightarrow Q, K, V \longrightarrow Z.$$

- each token can look at all other tokens;
- relationships are learned from data;
- computation can be parallelised across positions;
- long-range dependencies become easier.

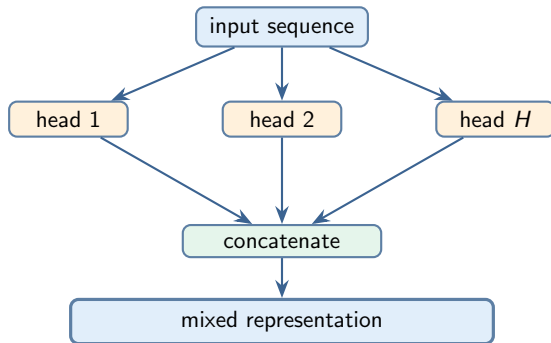


Multi-Head Attention

One attention operation may focus on one type of relationship. Multi-head attention runs several attention mechanisms in parallel.

$$\text{head}_r = \text{Attention}(\mathbf{QW}_Q^{(r)}, \mathbf{KW}_K^{(r)}, \mathbf{VW}_V^{(r)}).$$

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}_O.$$



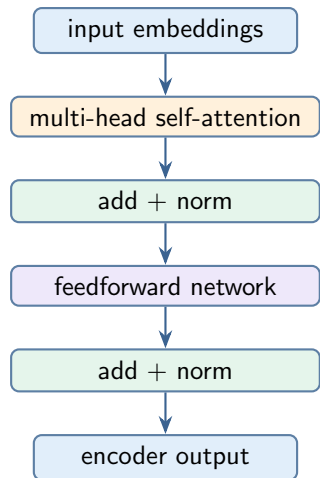
The Transformer Encoder Block

A transformer encoder is built by stacking repeated encoder blocks.

Each block contains:

- 1 multi-head self-attention;
- 2 residual connection and layer normalisation;
- 3 feedforward neural network;
- 4 residual connection and layer normalisation.

The block preserves sequence length but updates the representation at every position.



Position Information

Self-attention alone does not know the order of tokens.

attention sees a set of vectors unless we add position information.

Token embedding

Represents what the token is.

$\mathbf{e}_i$

Position embedding

Represents where the token is.

$\mathbf{p}_i$

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i.$$

Important point

Transformers do not process tokens one by one like RNNs. Order must be supplied explicitly.

Encoder, Decoder, and Causal Masking

Different transformer variants are used for different supervised tasks.

Encoder-only

Uses bidirectional context.

Good for classification and regression from a sequence.

Decoder-only

Predicts the next token.

Uses causal masking so future tokens are hidden.

Encoder-decoder

Maps one sequence to another.

Good for translation and sequence-to-sequence tasks.

causal mask: $\alpha_{ij} = 0$ for $j > i$.

Why Transformers Became So Important

Strengths

- capture long-range dependencies;
- train in parallel across sequence positions;
- scale well with data and model size;
- work for text, time series, images, graphs, and multimodal data.

Limitations

- attention cost grows with sequence length;
- large models require substantial compute;
- predictions can be hard to interpret;
- careful training and validation are essential.



Choosing a Model Family

Data structure	Natural model family	Reason
Tabular data	Trees, boosting, MLPs	Feature interactions and nonlinear effects
Images and grids	CNNs, vision transformers	Spatial structure and local patterns
Graphs	GNNs	Neighbourhood and connectivity structure
Time series	RNNs, LSTMs, transformers	Temporal dependence and context
Text and tokens	Transformers	Flexible long-range token interactions

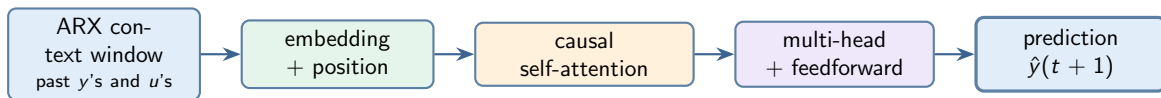
Final message

The best architecture is not chosen in isolation. It should match the data structure, the task, the dataset size, and the required level of interpretability.

Transformer for ARX-Type Modelling

We want to predict the next output from past outputs and past exogenous inputs:

$$\hat{y}(t+1) = f(y(t), y(t-1), \dots, u(t), u(t-1), \dots).$$

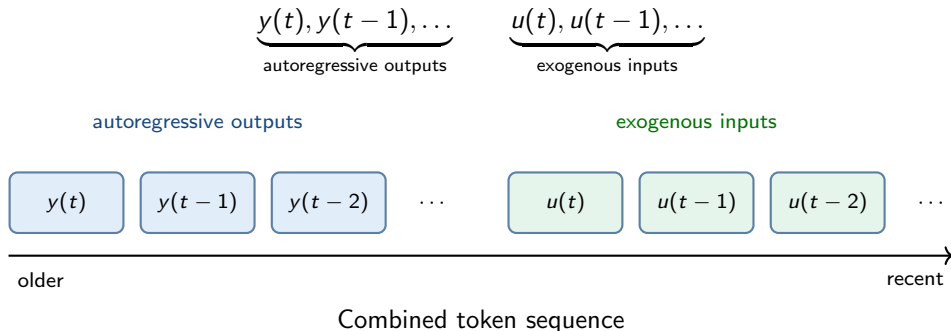


Idea

A transformer learns which past outputs and exogenous inputs are most relevant for the next prediction.

Input Tokens: Past Outputs and Exogenous Inputs

The ARX context window is converted into a sequence of tokens.



Embedding and Positional Encoding

Each token is mapped to a vector, and position information is added.

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i.$$

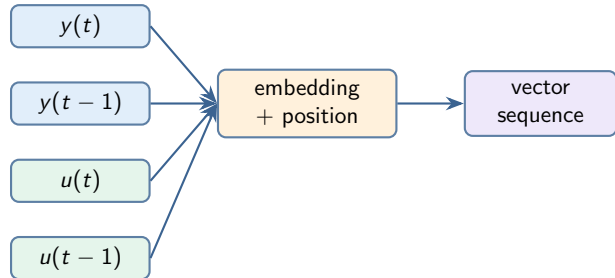
Token embedding

Represents the value and type of the token.

$$y(t - k) \quad \text{or} \quad u(t - k)$$

Position encoding

Represents where the token sits in the time window.



Queries, Keys, and Values for ARX Attention

From the embedded sequence, the transformer forms

$$\mathbf{Q} = \mathbf{XW}_Q, \quad \mathbf{K} = \mathbf{XW}_K, \quad \mathbf{V} = \mathbf{XW}_V.$$

Query

What is the current prediction position looking for?

$\mathbf{q}_i$

Key

What information does each past token offer?

$\mathbf{k}_j$

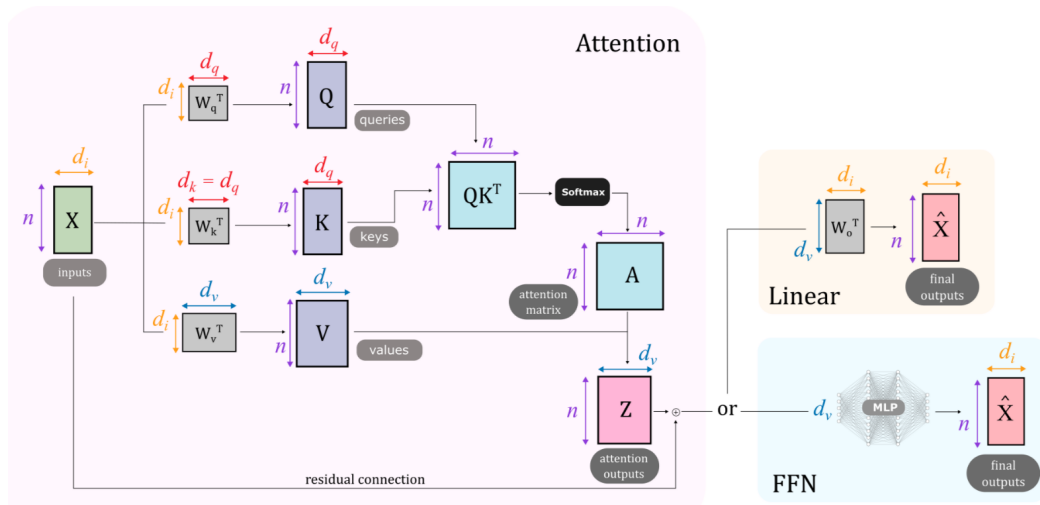
Value

What information is passed forward?

$\mathbf{v}_j$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

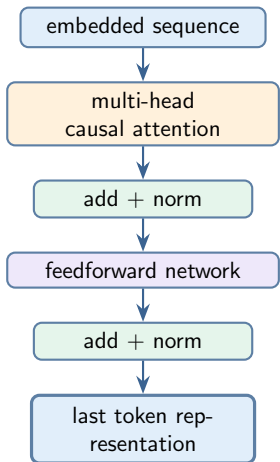
Multi-Head Attention for Dynamic Systems



by Gregory Duthé, Nikolaos Evangelou

Transformer Block and Output Prediction

The attention output is passed through a position-wise feedforward network and then projected to the prediction.



Use the representation of the most recent token:

$$\mathbf{z}_N$$

$$\hat{y}(t+1) = \mathbf{w}_{\text{out}}^T \mathbf{z}_N + b_{\text{out}}.$$

Final view

The transformer is a flexible nonlinear ARX model:

$$\hat{y}(t+1) = f_{\theta}(y_{\text{past}}, u_{\text{past}}).$$